

Critical-Path Profiler for Ray

Harsh Shah Neha A Purav Sanghvi

CS546 - raft-profiler

April 28, 2026

Abstract

Ray’s dashboard and tracing tools show task timelines and resource utilization, but they do not directly answer which tasks *determined* when a job finished. This project implements **raft-profiler**: a critical-path profiler for Ray Core that records data dependencies at task submission, reconstructs a task DAG, computes the longest causal chain, splits time into waiting vs. compute using State API events, and visualizes results in a self-contained HTML dashboard. Evaluation in this document is scoped to three benchmarks only: Ray batch prediction (shared loader vs. branch straggler), skewed analytics via Modin on Ray, and a Daft-on-Ray pipeline that stresses shuffle and UDF execution with a best-effort fallback when explicit `ObjectRef` edges are hidden.

1 Overview

Ray’s dashboard and tools such as Jaeger [9, 8] show task timelines and resource utilization. Neither answers the question that matters when a job is slow: **which tasks actually determined when the job finished?** Our setting is Ray Core [1, 2], which exposes a dynamic task graph rather than a purely stage-centric abstraction such as Spark’s RDDs.

This profiler computes the **critical path** of a Ray job: the longest chain of dependent tasks through the execution DAG. Tasks not on this chain have no impact on total runtime regardless of how slow they are (holding the DAG and scheduling model fixed). Tasks on the chain, if sped up, directly reduce job completion time.

The system integrates with existing programs through minimal setup at initialization and termination, requiring no modifications to application logic. The resulting analysis is exposed through a browser-based UI.

2 The five deliverables

The following deliverables match our initial project proposal; the implementation lives under `raft-profiler/profiler/` and `raft-profiler/benchmarks/`.

1. Records task timing and dependency information

Timing. After the job completes, task timing comes from the Ray State API. We call `list_tasks` with `detail=True`. The response includes creation, start, and end timestamps, plus a state transition log used for the breakdown in Deliverable 4.

Dependencies. Dependency information is recorded at submission time via a lightweight instrumentation layer. The layer wraps the task submission interface (`ray.remote`), inspects

every call’s arguments for `ObjectRef` values, records them as dependency edges, and forwards the submission unchanged.

Ray’s programming model makes this practical: a task that needs another task’s output must receive an explicit object reference, so dependency information is visible at the call site.

Unlike systems where trace context must be threaded manually [8, 7], this profiler extracts data-flow edges automatically from explicit references (including one level of unpacking for lists and tuples).

2. Reconstructs the job’s task dependency graph

The State API’s parent/spawn relationships record *who launched* a task, not necessarily *which outputs* it waited on. For example, a reducer waiting on ten mappers may appear with a single parent edge, not ten data-dependency edges. Using the edges captured in Deliverable 1, the profiler builds a directed acyclic graph in which each edge represents a true data dependency: the downstream task cannot start until the upstream task’s output is available.

Ray Tune, Ray AIR, and Ray Serve are outside the automatic scope for the same reason many frameworks hide internal submissions: they may generate tasks without exposing `ObjectRefs` at user code. Extending coverage would require hooks inside those libraries or native Ray instrumentation.

3. Identifies the critical path that determined total job runtime

The DAG is processed in topological order (Kahn’s algorithm [11]). For each task t , let x_t be execution duration from timestamps, s_t and e_t start and end times, and $P(t)$ resolved predecessor ids from the graph. The implementation selects a predecessor with maximum partial critical-path value; if **no** explicit predecessors exist, a **fallback heuristic** (for engines such as Daft on Ray [6]) links t to the most recent finishing task before s_t , so that visualization and path length remain informative when refs are hidden inside framework code.

Scheduling delay after the chosen predecessor p is $\delta_t = \max(0, s_t - e_p)$ (or relative to global job start if there is no predecessor). The implemented dynamic program is:

$$C_t = x_t + \delta_t + \max_{p \in P(t)} C_p, \quad \max_{p \in \emptyset} C_p = 0. \quad (1)$$

Equation (1) is a longest-path formulation in the spirit of classical critical-path methods [10]. Each task stores a pointer to the predecessor that achieved the maximum. The terminal task maximizes C_t ; the path is reconstructed by following pointers backward. Under the explicit-ref DAG model, slowdown along this chain maps to increased end-to-end time; under the fallback heuristic, paths are best-effort interpretations of temporal ordering, not proofs of logical dependence.

4. Breaks down total runtime into categories

The Ray State API provides a state transition log for tasks. The module `runtime.breakdown.py` parses event records, finds the transition into a running state, and attributes time before that boundary to **waiting** and time after to **computation**. This reduces reliance on hard-coded state name strings where possible. The split is intentionally coarse; finer attribution (e.g., transfer vs. queueing) depends on stable semantics of internal states and is left as future work.

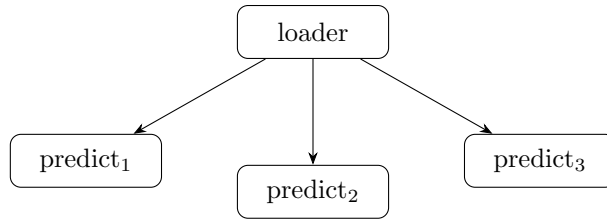


Figure 1: Batch prediction pattern: one loader feeds parallel predictors via explicit `ObjectRefs`; slowing the loader makes it a shared upstream bottleneck.

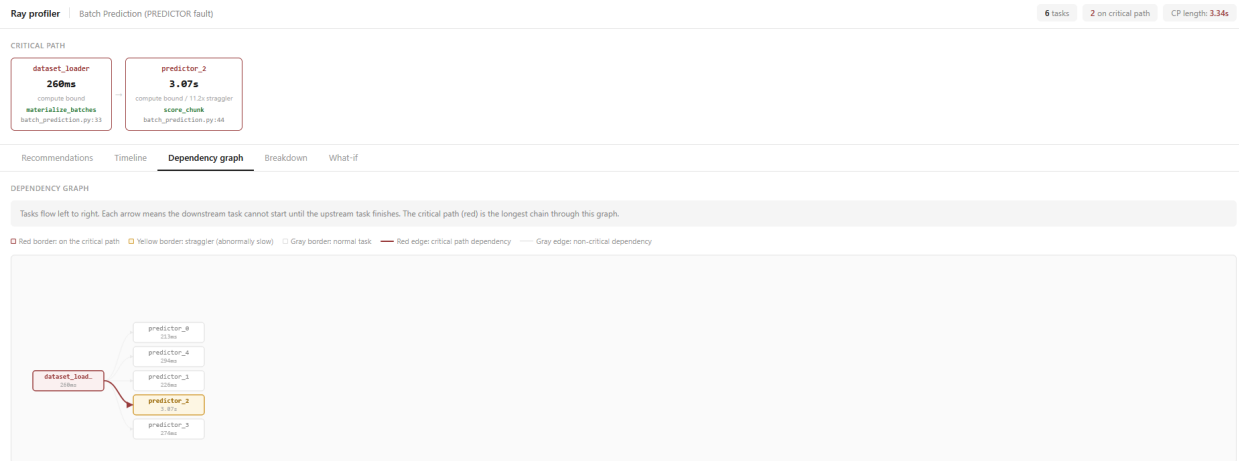


Figure 2: Raft-profiler HTML dashboard for the batch prediction benchmark.

5. Visualizes the critical path in an interactive UI

The profiler outputs a self-contained HTML dashboard (no application server required). It includes a timeline with waiting vs. compute segments, a force-directed DAG with critical-path highlighting, and a sortable breakdown table. Data are embedded as JSON in the page.

3 Implementation summary

Integration is two calls around existing Ray code: `profile()` immediately after `ray.init()` (before remote tasks are defined or invoked), and `print_critical_path(...)` at the end to query the State API, compute the path, and write HTML. Modules: `profiler.py` (dependency capture), `critical_path.py` (topological order + DP), `runtime_breakdown.py` (waiting/compute), `dashboard.py` (HTML).

4 Benchmarks and correctness

Scope. This report evaluates **only** the following three benchmarks, as requested for the course artifact.

4.1 Batch prediction

Reference: Ray batch prediction example. [docs.ray.io batch prediction](https://docs.ray.io/en/latest/batch-prediction.html)

Workload: `benchmarks/batch_prediction.py`. A loader task feeds data chunks to parallel predictor tasks via explicit `ObjectRefs`. The loader is a shared upstream bottleneck: slowing it blocks all downstream predictors and collapses effective parallelism. This contrasts with fan-in map-reduce patterns where only one slow branch need dominate.

Fault injection: A fixed `time.sleep` delay is injected into (1) the loader and (2) one predictor at a time (see script flags).

Expectation: When the loader is slowed, the critical path should run through the loader and downstream merge points that depend on it. When a single predictor is slowed, only that predictor’s branch should dominate the path. The profiler must distinguish a **shared bottleneck** from a **single-branch** bottleneck in the dashboard and CP listing.

4.2 Modin (skewed groupby)

Workload: `benchmarks/modin_skew.py`. Skewed categories concentrate work in one group; Modin [4, 5] executes the query on Ray. Dependencies are captured where Modin/Ray passes `ObjectRefs` in line with Ray Core semantics.

Expectation: The critical path should highlight tasks associated with the straggling partition or stage that processes the heavy group. The HTML timeline should show whether tail latency is waiting-dominated or compute-dominated for those tasks.

4.3 Daft on Ray

Workload: `benchmarks/daft-run.py`. A Daft dataframe runs on Ray with a heavy UDF and a sort (shuffle). Internal actors or runners may not surface user-level `ObjectRef` edges across every boundary.

Expectation: Where explicit edges exist, the DAG should reflect them. Where they do not, Deliverable 3’s **fallback heuristic** may synthesize edges from finish-to-start gaps; results should be interpreted as diagnostic hints, not as ground-truth logical dependence. The dashboard still surfaces which Ray tasks bounded wall time in practice.

Table 1: Run log

Benchmark	Wall (s)	#Tasks	CP length (s)
<code>batch_prediction.py</code> (loader fault)	10	6	3.34
<code>batch_prediction.py</code> (predictor fault)	10	6	3.34
<code>modin_skew.py</code>	20	12	6.09
<code>daft-run.py</code>	28	8	7.12

5 Limitations

Ray Core emphasis. The profiler instruments task submission by intercepting `ObjectRef` arguments at the call site. Ray Tune, Ray AIR, and Ray Serve may submit work internally without user-visible refs, so the dependency graph cannot always be reconstructed automatically.

Post-hoc collection. Timing is collected after the job finishes via the State API snapshot, not as a live stream; there is no in-flight critical path view.

Nested arguments. Dependencies inside arbitrary dicts or deeply nested structures are not fully walked; lists and tuples are unpacked one level.

Heuristic edges. For Daft-style workloads, synthetic edges from the fallback heuristic may misrepresent true logical dependencies if unrelated tasks align in time; third-party frameworks may also overlap driver-side tasks with worker compute so the critical-path highlight can differ from the heaviest kernel in wall-clock terms.

6 Conclusion

`raft-profiler` implements the five proposal deliverables: capture, DAG reconstruction, critical path, waiting/compute breakdown, and HTML visualization. It validates them on batch prediction, Modin skew, and Daft-on-Ray benchmarks. Future work includes richer argument walking (with safeguards), stronger integration with Ray’s internal event semantics, and optional support for higher-level Ray libraries if submission hooks become available.

Acknowledgments

Ray, Modin, and Daft are third-party systems; this project builds on their documentation and APIs.

References

- [1] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica, “Ray: A Distributed Framework for Emerging AI Applications,” in *Proc. USENIX OSDI*, 2018.
- [2] Ray Team, *Ray Documentation* (Ray Core, scheduling, State API, batch prediction), <https://docs.ray.io/>
- [3] C. Curtsinger and E. D. Berger, “COZ: Finding Code that Counts with Causal Profiling,” in *Proc. USENIX ATC*, Denver, CO, 2016. <https://www.usenix.org/conference/atc16/technical-sessions/presentation/curtsinger>
- [4] D. Petersohn, S. Macke, D. Xin, W. Ma, D. Lee, X. Mo, J. E. Gonzalez, J. M. Hellerstein, A. D. Joseph, and A. Parameswaran, “Towards Scalable Dataframe Systems,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 2033–2046, 2020.
- [5] Modin Development Team, *Modin Documentation*, <https://modin.readthedocs.io/>
- [6] Eventual Inc., *Daft: Documentation and User Guide*, <https://docs.daft.ai/>
- [7] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google Technical Report, 2010.
- [8] OpenTelemetry Project, “OpenTelemetry Specification,” Cloud Native Computing Foundation, <https://opentelemetry.io/docs/specs/>

- [9] Y. Shkuro, “Jaeger: Open source end-to-end distributed tracing,” 2017. <https://www.jaegertracing.io/>
- [10] J. E. Kelley, Jr. and M. R. Walker, “Critical-path planning and scheduling,” in *Proc. Eastern Joint Computer Conference*, pp. 160–173, 1959.
- [11] A. B. Kahn, “Topological sorting of large networks,” *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.